ROUTEGUARD

# Submission Checklist & Report Plan

*RouteGuard — C11, syllabus-aligned scope*

Companion to RouteGuard_SRS_v0.2
Based on the supplied Google Form screenshot and course handouts
Use as a working checklist; mark items only when verified

**IMPORTANT STATUS NOTE**

Verify implementation and test evidence before claiming completion. The project engine must be C11 and remain within the Lecture 37 boundary.

**Version:** 0.2 — aligned to C11 and CSC2101 through Lecture 37

**Status:** Preparation guide; items are not complete until verified

**Project status:** Proposed; implementation/test evidence must be added as it is produced

## A. Fields required by the supplied Google Form screenshot

- ☐ **Project Title:** RouteGuard: Graph-Based Route Discovery and Network Connectivity Simulator (confirm final title with team/faculty).
- ☐ **Project Report (PDF):** one PDF, maximum 10 MB.
- ☐ **Project Presentation:** one PDF or presentation file, maximum 10 MB.
- ☐ **GitHub Repository Link:** public/accessible repository URL if public visibility is chosen; verify that the URL opens the intended project.

The form screenshot indicates the Google account name, email address, and photo associated with the account may be recorded when uploading/submitting. Use the intended account and retain the submission confirmation/response email.

## B. Project specification constraints

- ☐ Algorithm engine is written in **C11**, not C++.
- ☐ Core topics match CSC2101 classes through Lecture 37: graph representation, BFS, DFS, path reachability/minimum-hop path using BFS, and complexity analysis.
- ☐ Lab plan alignment: CSC2130 uses C; lab Class 9 covers adjacency matrix/list and Class 10 covers BFS/DFS.
- ☐ No Dijkstra, A*, greedy shortest path, low-link bridge/articulation algorithms, or other post-boundary algorithm is included without explicit faculty approval.
- ☐ The graph is unweighted, so route length means number of edges/hops—not physical distance or travel time.
- ☐ Edge closure simulation uses BFS/DFS to recalculate reachability/connectivity; do not overstate it as a dedicated advanced resilience algorithm.
- ☐ SRS, README, report, slides, and repository description all agree on the same scope and language.

## C. Repository and reproducibility

- ☐ Repository includes C source, public headers, test sources, canonical synthetic fixture, build configuration, README, and SRS.
- ☐ No passwords, tokens, personal data, unrelated files, native executables, temporary object files, or dependency directories are committed.
- ☐ README lists actual prerequisites and exact build/run/test commands.
- ☐ A clean build/test was run from the final committed source.
- ☐ Fresh-clone reproducibility is claimed only if actually tested.
- ☐ If a React/WASM interface is included, native C tests and WASM/browser tests are reported separately.
- ☐ The exact repository URL in the form points to the final branch and includes required source/assets.

## D. Correctness and test evidence

- ☐ Matrix and adjacency list represent the same undirected graph and active-edge state.
- ☐ BFS traversal order matches a deterministic hand-checked fixture.

- ☐ BFS route has the minimum number of edges, with correct source/destination and predecessor reconstruction.
- ☐ DFS traversal/reachability is correct.
- ☐ Connected-component counts match known fixtures.
- ☐ Disabling an edge correctly changes traversal/reachability; re-enabling and reset restore expected results.
- ☐ Source-equals-destination, unreachable destination, isolated vertices, cycles, and invalid IDs are tested.
- ☐ Operation counters have explicit definitions and deterministic assertions; elapsed time is informational.
- ☐ Test counts, assertions, outputs, and complexity claims match actual execution evidence.
- ☐ Any unsupported/unrun tests and sanitizer limitations are stated accurately.

## E. Final project report PDF

- ☐ PDF includes title page, course/context, authorship fields required by faculty, abstract, problem statement, objectives, scope, SRS, design, algorithms, implementation, tests, results, limitations, conclusion, and references.
- ☐ The report explicitly identifies C11 and the Lecture 37 scope boundary.
- ☐ It explains matrix versus list representation and their storage/traversal complexity.
- ☐ It explains BFS as minimum-hop path search for an unweighted graph, and DFS as traversal/reachability (not a shortest-path guarantee).
- ☐ At least one actual example shows a baseline route, a disabled edge, and the recomputed result.
- ☐ Screenshots come from the actual running application and correspond to the tested code version.
- ☐ No fabricated benchmarks, mocked output presented as runtime evidence, clipped tables, blank pages, or editing artifacts remain.
- ☐ PDF opens and is no larger than 10 MB.

## F. Presentation

- ☐ Start with the problem: route discovery and connectivity when a pathway is unavailable.
- ☐ Show vertices, undirected edges, active/disabled edges, source, and destination.
- ☐ Explain BFS queue behavior and why it finds a minimum-hop route in an unweighted graph.
- ☐ Explain DFS reachability and why its returned path is not necessarily shortest.
- ☐ Compare adjacency matrix and adjacency list space/time trade-offs.
- ☐ Demonstrate an edge closure, rerun traversal, show whether the destination remains reachable, and restore the graph.
- ☐ Explain operation counters and asymptotic complexity through Lecture 37.
- ☐ Avoid presenting Dijkstra, weighted routing, bridges/articulation points, or greedy methods as implemented/covered within the constraint.
- ☐ Prepare screenshots or a recording as a fallback for live-demo failures.
- ☐ Presentation file opens and is no larger than 10 MB.

## G. Required form submission confirmation

- ☐ Final project title consistent across form, repository, report, and slides.
- ☐ Project report PDF uploaded in the correct field.
- ☐ Project presentation uploaded in the correct field.
- ☐ GitHub link pasted and opened once to verify it is correct.

☐ Required fields completed.

☐ Submit once, wait for confirmation, and retain the response receipt/email.

## H. Recommended detailed report structure

1. **Cover page** — title, student/team names and IDs as required, course code, faculty, institution, term.
2. **Abstract** — problem, C11 solution, graph model, core algorithms, and verified result.
3. **Introduction and motivation** — model a small network where links may become unavailable.
4. **Problem statement** — find a minimum-hop route and determine reachability when connections are closed.
5. **Objectives and scope** — explicitly state the Lecture 37 cutoff and non-goals.
6. **Syllabus mapping** — CSC2101 classes 23–28 and 33–37; CSC2130 lab Classes 9–10.
7. **SRS** — user requirements, functional/non-functional requirements, constraints, acceptance criteria.
8. **Architecture** — C11 graph engine and optional separate presentation layer.
9. **Graph model and representations** — vertex/edge model; adjacency matrix versus adjacency list.
10. **Algorithm design** — BFS traversal, BFS minimum-hop route reconstruction, DFS traversal/reachability, connected components, edge toggling.
11. **Complexity analysis** — table for time/space cost with V and E definitions.
12. **Implementation details** — C files/modules, memory management, data invariants, UI bridge only if implemented.
13. **Testing** — deterministic fixtures, expected/actual results, edge-case coverage, build commands.
14. **Results and discussion** — actual outputs/screenshots and a closure scenario; explain any differences across representations.
15. **Limitations and future work** — separate unimplemented features from delivered work; do not quietly add out-of-scope algorithms.
16. **Conclusion, references, appendix** — verified work, source handouts, and additional test evidence if required.

## I. SRS-to-implementation traceability

| Requirement group | Source file/function | Test evidence | Status |
|---|---|---|---|
| Graph validation and matrix/list | | | Not started |
| BFS traversal | | | Not started |
| BFS minimum-hop route | | | Not started |
| DFS and reachability | | | Not started |
| Connected components | | | Not started |
| Edge disable/enable/reset | | | Not started |
| Operation counters/complexity display | | | Not started |

| Requirement group | Source file/function | Test evidence | Status |
|---|---|---|---|
| Frontend (if included) | | | Not started |
| Native tests | | | Not started |
| WASM/browser integration (if included) | | | Deferred |
| Fresh-clone workflow | | | Not started |

Allowed status values: Not started, In progress, Implemented, Tested, Deferred. Use Tested only when there is an actual passing test or recorded acceptance check.

## J. Source documents used

1. Manipal University Jaipur, **CSC2101 — Data Structures and Algorithms, Course Handout, July–December 2026**, Section G, Lecture Plan: classes 23–28 and 33–37.
2. Manipal University Jaipur, **CSC2130 — Data Structures and Algorithms Lab, Course Handout, July–December 2026**, Section B and Section G: C language; lab Classes 9–10.

If the instructor provides a project rubric/report template, that document takes precedence over this suggested report outline.